

Security Audit

Onramp (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	26
Conclusion	27
Our Methodology	28
Disclaimers	30
About Hashlock	31

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Onramp team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Hann Finance is positioning itself as a stability-first DeFi platform built for the KAIA ecosystem, targeting users familiar with web3 liquidity-markets ("Kaia degens"). The website invites users to join a waitlist and hints that its testnet is already live. It emphasises simplicity and robustness, promising a next-generation source of liquidity with minimal complexity.

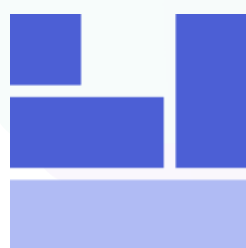
Project Name: Hann Finance

Project Type: Defi

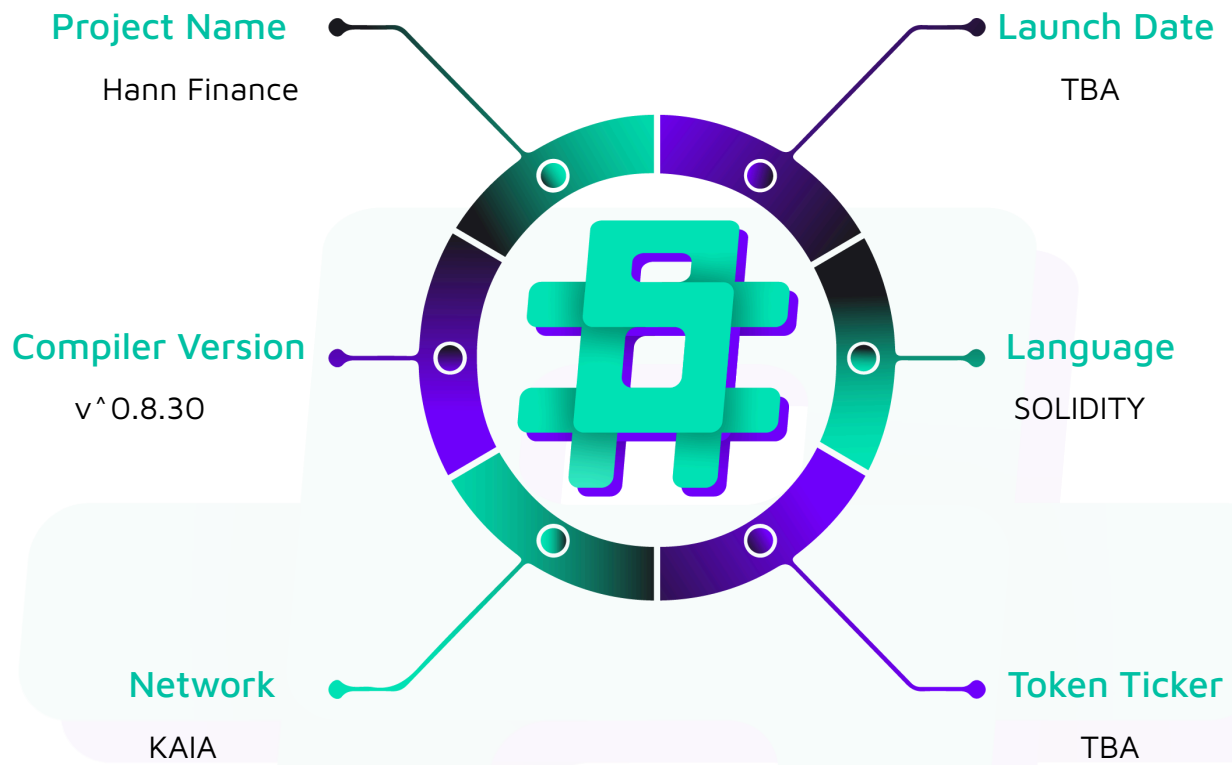
Compiler Version: ^0.8.30

Website: <https://hann.finance>

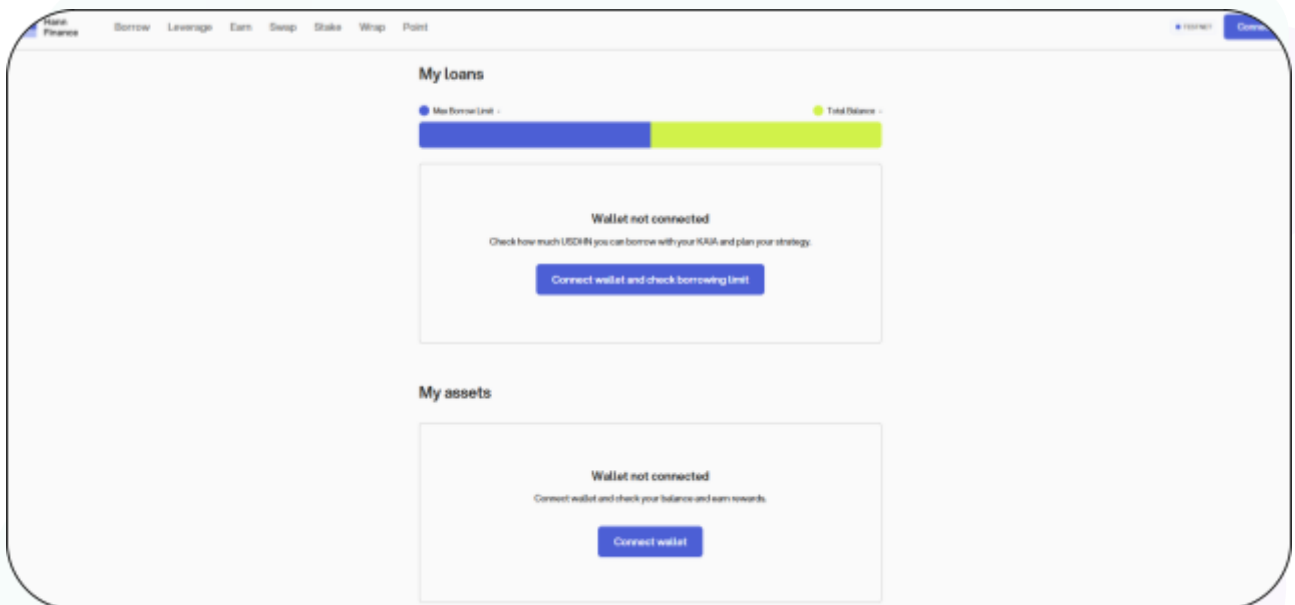
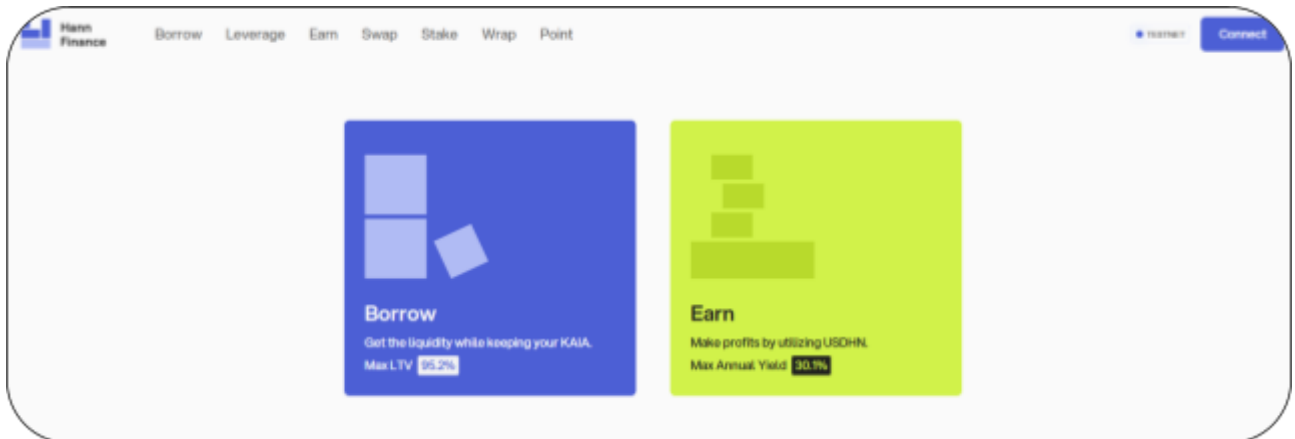
Logo:



Hann Finance

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Hann Finance project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Hann Finance Smart Contracts
Platform	KAIA / Solidity
Audit Date	November, 2025
GitHub link2	https://github.com/Hann-Finance/cdp-core
Contract 1	Interfaces/IStableSwap.sol
Contract 2	Interfaces/IStableSwapFactory.sol
Contract 3	Interfaces/IStableSwapRouter.sol
Contract 4	permit2/IAllowanceTransfer.sol
Contract 5	permit2/ISignatureTransfer.sol
Contract 6	permit2/LPToken.sol
Contract 7	permit2/StableSwap.sol
Contract 8	permit2/StableSwapFactory.sol
Contract 9	permit2/StableSwapRouter.sol
Audited GitHub Commit Hash	4a0e87326f3c44cdda00bf4376ac9c366585bf60
Fix Review GitHub Commit Hash	de85b0476118dbefae38087349cfdef9ee0cfe6b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section, and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 High severity vulnerabilities

3 Medium severity vulnerabilities

4 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
CDP-Core(StableSwap) • Core Automated Market Maker (AMM): Implements a two-token stablecoin AMM, similar to Curve's StableSwap, designed for low-slippage trades between pegged assets. • Dynamic Amplification: Features a configurable "amplification" parameter that can be adjusted or gradually ramped to optimize the trade-off between slippage and liquidity depth. • Liquidity Provisioning & LP Tokens: Allows users to deposit pairs of tokens to provide liquidity and receive ERC20 LP (Liquidity Provider) tokens in return, which represent their share of the pool. • Multi-hop Trade Routing: Includes a router contract that enables complex trades across multiple StableSwap pools in a single transaction. • Factory-based Deployment: Utilizes a factory pattern to deploy new, standardized StableSwap pools and their corresponding LP tokens. • Gasless Approvals via Permit2: Integrates with Uniswap's Permit2 contract to allow for gas-efficient, signature-based token approvals, enhancing user experience.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Hann Finance project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Hann Finance project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] StableSwapRouter#swapTokensForExactTokens, StableSwapRouter#_swapExactOutPath - amounts array not updated in multi hop swaps

Description

The multi hop exact output swap functions could fail because the `amounts` array is pre calculated but never updated with the actual output from each swap hop, causing subsequent hops to use quoted amounts instead of actual received amounts.

Vulnerability Details

In `swapTokensForExactTokens` and `_swapExactOutPath`, the code pre calculates the `amounts` array using `_quoteAmountsIn()` but never updates `amounts[i + 1]` with the `actualOut` value returned from each swap execution.

```
uint256 actualOut = pool.swap(path[i].tokenIn, amounts[i], minOut, recipient, deadline);
```

When a multi-hop swap executes:

1. Hop 1 swaps `amounts[0]` tokens and receives `actualOut` tokens
2. The `amounts[1]` value remains the pre-calculated quoted amount, not the actual output
3. Hop 2 attempts to swap `amounts[1]` tokens, but the router only holds `actualOut` tokens from hop 1

If `actualOut < amounts[i + 1]`: The next hop tries to swap more tokens than available, causing the ERC20 transfer to revert and the transaction could fail.

If `actualOut > amounts[i + 1]`: The next hop only uses the quoted amount, leaving excess tokens (dust) permanently locked in the router contract.

Impact

Multi-hop exact output swaps could fail when actual output differs from quoted amounts, and excess tokens could become permanently locked in the router when actual output exceeds quoted amounts.

Recommendation

Update the `amounts` array with the actual output after each swap, similar to how `swapExactTokensForTokens` and `_swapExactInPath` handle it.

Status

Resolved

Medium

[M-01] `StableSwapFactory#createPoolWithFees`, `StableSwap#initialize` - Mismatch between the factory's default fee and hardcoded values in the pool initialization

Description

When the factory owner changes default fees and pool deployers use the new default values in `createPoolWithFees()`, the conditional logic fails to apply the intended fees, resulting in pools being initialized with hardcoded values instead of the specified fees.

Vulnerability Details

The `StableSwap.initialize()` method sets hardcoded fee values that match the factory's initial defaults. However, when the factory owner updates the default fees in the factory contract using `setDefaultSwapFee()` and `setDefaultAdminFee()`, these hardcoded values become stale.

The `createPoolWithFees()` function in `StableSwapFactory` only calls `setSwapFee()` and `setAdminFee()` when the provided fees differ from the current factory defaults:

```
if (swapFee != defaultSwapFee) {  
    StableSwap(pool).setSwapFee(swapFee);  
}  
  
if (adminFee != defaultAdminFee) {  
    StableSwap(pool).setAdminFee(adminFee);  
}
```

This creates a problematic scenario:

1. Owner calls `setDefaultSwapFee(8)` and `setDefaultAdminFee(3000)`
2. Pool deployer calls `createPoolWithFees(tokenA, tokenB, 8, 3000)` using the new defaults
3. The conditions `if (8 != 8)` and `if (3000 != 3000)` evaluate to false

4. Neither `setSwapFee()` nor `setAdminFee()` are called
5. The pool retains the hardcoded values: `swapFee = 4` and `adminFee = 5000`

The issue arises from the mismatch between the hardcoded values in `initialize()` and the factory's current default values, combined with conditional logic that assumes the pool starts with the factory's current defaults.

Impact

Pools created with updated default fees will silently use incorrect fee structures.

Recommendation

Consider the following options:

- Modify `initialize()` to accept fee parameters and set them directly
- Do not set fees in `initialize()` and always call `setSwapFee()` and `setAdminFee()` in the factory

Status

Resolved

[M-02] StableSwapRouter#_validatePool - Weak pool validation allows bypass with malicious contracts

Description

The `_validatePool()` function only checks if the pool's factory matches the router's factory, which can be bypassed by any contract that implements a `factory()` function returning the expected factory address.

Vulnerability Details

The `_validatePool()` function in `StableSwapRouter` performs a weak validation check:

```
function _validatePool(address pool) private view returns (IStableSwap stableSwap) {  
    stableSwap = IStableSwap(pool);  
    if (stableSwap.factory() != FACTORY) revert InvalidPool();  
}
```

This validation only verifies that the pool's `factory()` function returns the same address as `FACTORY`. However, any malicious contract can implement the `IStableSwap` interface with a `factory()` function that returns `FACTORY`, effectively bypassing this validation.

Impact

Malicious contracts can bypass pool validation and be used in router operations, potentially leading to unexpected behavior or fund loss.

Recommendation

Replace the factory check with a proper validation that verifies the pool is registered in the factory:

```
function _validatePool(address pool) private view returns (IStableSwap stableSwap) {  
    stableSwap = IStableSwap(pool);  
    address token0 = stableSwap.token0();  
    address token1 = stableSwap.token1();
```

```
        if (IStableSwapFactory(FACTORY).getPool(token0, token1) != pool) revert  
InvalidPool();  
    }
```

Status

Resolved



[M-03] StableSwap#quoteAddLiquidity - First deposit quote doesn't account for minimum liquidity burn

Description

The `quoteAddLiquidity` function returns the full invariant `D1` as the expected liquidity for the first deposit, but `addLiquidity` actually mints `D1 - STABLESWAP_MINIMUM_LIQUIDITY` to the user. This discrepancy causes legitimate transactions to fail when users set `minLiquidity` equal to the quoted value.

Vulnerability Details

When `totalSupply == 0` (first deposit), `quoteAddLiquidity` calculates and returns the full invariant `D1`:

```
if (totalSupply == 0) {
    // Initial deposit: no fee on imbalance

    uint256 D1 = StableMath.getD(_currentA(), reserve0 + normalized0, reserve1 +
normalized1);

    liquidity = D1;

    if (liquidity <= STABLESWAP_MINIMUM_LIQUIDITY) revert InvalidAmount();
}
```

However, in `addLiquidity`, the `_computeAddLiquidity` function returns `D1 - STABLESWAP_MINIMUM_LIQUIDITY` for the first deposit:

```
if (cache.mintMinimum) {
    uint256 invariantAfterDeposit =

        StableMath.getD(Aeff, reserve0 + cache.normalized0, reserve1 +
cache.normalized1);

    if (invariantAfterDeposit <= STABLESWAP_MINIMUM_LIQUIDITY) revert InvalidAmount();

    liquidity = invariantAfterDeposit - STABLESWAP_MINIMUM_LIQUIDITY;

    return (liquidity, 0, 0);
}
```

```
}
```

Then `addLiquidity` mints `STABLESWAP_MINIMUM_LIQUIDITY` to `address(0)` and the remaining `liquidity` to the user:

```
if (cache.mintMinimum) {
    LPToken(lpToken).mint(address(0), STABLESWAP_MINIMUM_LIQUIDITY);
} else {
    _accrueDepositAdminFees(cache.fee0, cache.fee1);
}

if (liquidity < minLiquidity) revert SlippageExceeded();

LPToken(lpToken).mint(to, liquidity);
```

If a user calls `quoteAddLiquidity` and sets `minLiquidity` equal to the returned quote, the transaction will revert with `SlippageExceeded` because the actual minted liquidity is less than the quoted value.

Impact

Users who rely on `quoteAddLiquidity` to set their `minLiquidity` parameter for the first deposit will experience transaction failures, preventing them from adding initial liquidity to the pool.

Recommendation

Modify `quoteAddLiquidity` to return `D1 - STABLESWAP_MINIMUM_LIQUIDITY` for the first deposit case to match the actual liquidity minted to the user.

Status

Resolved

Low

[L-01] LPToken#permit - Signature malleability vulnerability in ecrecover implementation

Description

The permit function directly uses ecrecover that is vulnerable to signature malleability attacks, which could allow attackers to replay signatures with modified parameters.

Vulnerability Details

The permit function uses ecrecover to validate signatures without implementing checks for signature malleability. It has no validation that the s value is in the lower half of the curve's order. While the nonce mechanism prevents signature replay attacks, the lack of proper ECDSA checks remains a deviation from cryptographic best practices.

Impact

Signature malleability could allow attackers to replay signatures, potentially bypassing intended security controls and authorization mechanisms.

Recommendation

It is recommended to enforce that the s value of the signature is in the lower half of the elliptic curve order. This can be achieved by using a library like OpenZeppelin's [ECDSA.sol](#) for signature recovery and verification instead of the native ecrecover precompile.

Status

Resolved

[L-02] All contracts - Floating pragma

Description

Contracts use floating pragma statements `pragma solidity ^0.8.24` in cdp-core and `pragma solidity ^0.8.30` Ist-core instead of fixed version pragmas.

Vulnerability Details

Floating pragmas allow contracts to be compiled with different compiler versions than those tested, potentially introducing unidentified security issues.

Impact

Different pragma versions in test and production may pose unidentified security issues. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

Recommendation

Consider locking the pragma version to the specific version in each repository `pragma solidity 0.8.24` and `solidity 0.8.30`

Status

Resolved

[L-03] StableSwapFactory#createPoolWithFees - Pool creation is vulnerable to frontrunning

Description

The `createPoolWithFees()` function is vulnerable to frontrunning attacks, allowing malicious actors to observe pool creation transactions in the mempool and frontrun them with different fee parameters, preventing legitimate users from creating pools with their desired fees.

Vulnerability Details

The `createPoolWithFees()` function in `StableSwapFactory` allows anyone to create a pool with custom fees. However, since pool creation transactions are visible in the mempool before being mined, malicious actors can frontrun legitimate pool creation attempts.

The attack scenario works as follows:

1. Alice wants to create a tokenA/tokenB pool with 0.01% swap fee (1 basis point) and 10% admin fee (1000 basis points)
2. Alice submits a transaction calling `createPoolWithFees(tokenA, tokenB, 1, 1000)`
3. Bob observes this transaction in the mempool
4. Bob frontruns Alice by submitting a transaction with higher gas price calling `createPoolWithFees(tokenA, tokenB, 4, 5000)` with different fees
5. Bob's transaction gets mined first, creating the pool with his chosen fees
6. Alice's transaction fails with `PoolAlreadyExists()` revert
7. Alice has no way to create the pool with her desired fees

Once a pool is created, the only way to update the pool fees is through the factory owner calling `setPoolFees()`, which requires owner privileges. This means that if a pool is frontrun and created with undesired fees, the original creator cannot update the fees themselves and must rely on the factory owner to correct them.

Impact

Legitimate users can be prevented from creating pools with their desired fee parameters, forcing them to either accept pools created by frontrunners with potentially unfavorable fees or rely on the factory owner to update fees after creation.

Recommendation

There is no easy way to mitigate this given the permissionless design of the factory. Consider documenting this behavior, and if a pool is created with undesired fees, communication between the teams related to the tokens and the owner of the factory can be coordinated off-chain to update them.

Status

Resolved

[L-04] **StableSwapFactory#setFeeToSetter, StableSwapFactory#pause, StableSwapFactory#unpause** - Missing events

Description

The `setFeeToSetter()`, `pause()`, and `unpause()` functions in `StableSwapFactory` do not emit events when executing state changes, making it difficult to track these operations off-chain and reducing transparency for users and monitoring systems.

Vulnerability Details

The `StableSwapFactory` contract implements several administrative functions that modify state variables, but three of these functions fail to emit events:

- `setFeeToSetter()`
- `pause()`
- `unpause()`

Impact

Off-chain monitoring systems and users cannot efficiently track state changes

Recommendation

Add event emissions to these functions following the existing pattern in the contract.

Status

Resolved

Centralisation

The Hann Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the Hann Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.